CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 14: Matching

Harvard SEAS - Spring 2026 *2026-03-24*

1 Announcements

- Anurag OH after class.
- Pset 6 released.
- Pset 5 due tomorrow.
- A correction to Lec 13 on Ed.
- Reminder: Cite LLM use in pset (if any).

Recommended Reading:

- Hesterberg–Vadhan 14.1-14.3
- CLRS 25.1

2 Loose end: BFS Coloring

The performance of greedy algorithms can be very sensitive to the order in which decisions are made, and often we can achieve much better performance by picking a careful ordering. For example, we can process the vertices in *BFS order*, as described below.

```
BFSColoring(G):
Input  : A connected graph G = (V, E)
Output : A coloring f of G using "few" colors
Fix an arbitrary start vertex v_0 ∈ V;
Start breadth-first search from v_0 to obtain a vertex order v_1, v_2, ..., v_{n-1};
foreach i = 0 to n − 1 do
    f(v_i) = min {c ∈ ℕ : c ≠ f(v_j)  ∀j < i s.t. {v_i, v_j} ∈ E}.
return f
```

Algorithm 2.1: Greedy coloring in BFS order.

Running the `BFSColoring` algorithm we obtain

Now, we show that, in general, for 2-colorable graphs, `BFSColoring` finds a 2-coloring efficiently.

Theorem 2.1. *If G is a connected 2-colorable graph, then* `BFSColoring`(G) *will color G using 2 colors.*

Proof sketch.

□

Corollary 2.2. Graph 2-Coloring *can be solved in time $O(n + m)$.*

Proof.

□

3 The Computational Problem

A motivating problem. In the US, there is a nationwide system for arranging kidney transplants, matching patients (who need a kidney) with donors (who are willing to donate a kidney). Each donor can only donate one kidney (they need their other one to survive!) and only to certain patients (due to blood type and HLA type compatibilities). There over 100,000 patients currently on the kidney waiting list in the US, with a little over 25,000 donations happening per year, and patients spending an average of about 3.6 years on the waiting list. Algorithms like what we will cover in this chapter play a significant role in saving patients through kidney donations.

Q: How can we model kidney exchange graph-theoretically?

Definition 3.1. For a graph $G = (V, E)$, a *matching* in G is a subset $M \subseteq E$ such that _____ .

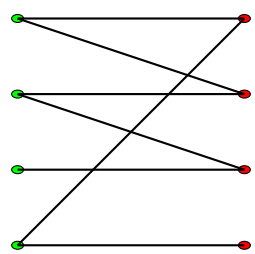
If a vertex v is incident to an edge in M, we say v is *matched* by M; otherwise we say it is *unmatched*.

We will focus on problem of finding the largest matching in a graph, called Maximum Matching.

Input: A graph $G = (V, E)$
Output: A matching $M \subseteq E$ in G of maximum size

Computational Problem Maximum Matching

Let's find a maximum matching in the graph below.



Q: What does our modeling omit? Specifically, what additional structure do kidney exchange compatibility graphs have that's not captured in Maximum Matching?

Fact: Maximum Matching can be efficiently reduced to Independent Set. (See textbook.) Why is this not so useful?

4 Alternating and Augmenting Paths

We might try to find a maximum matching by a *greedy strategy,* but this may not find the maximum-size matching. Let's see how it works on the example above (with a particular ordering of edges):

The greedy strategy is not unsalvageable, but we must add an operation more sophisticated than adding a single nonconflicting edge. To do so, we first introduce two graph theory notions related to matchings: *alternating walks* and *augmenting paths*.

Definition 4.1. Let $G = (V, E)$ be a graph, and M be a matching in G. Then:

1. An *alternating walk* W in G with respect to M is

2. An *augmenting path* P in G with respect to M is

Let's see an augmenting path with respect to the maximal (but not maximum) matching we found via the greedy algorithm:

Notice that if for every edge e in the augmenting path, we "switch" whether or not we include e in the matching, we get a strictly larger matching:

This suggests a natural algorithm for maximum matching: repeatedly try to find an augmenting path and use it to grow our matching. But we need to argue that augmenting paths always exist and we can find them efficiently. To turn this idea into an algorithm for finding maximum matchings, we need two things:

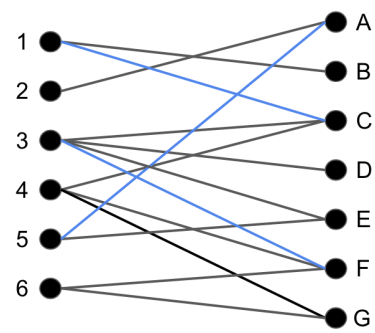
1.

2.

The former is the focus of Section 5 below. We can get the latter guarantee from the following theorem, for which a proof is in the textbook.

Theorem 4.2 (Berge's Theorem)**.** *Let $G = (V, E)$ be a graph, and $M \subseteq E$ be a matching. If (and only if) M is not a maximum-size matching, then (and only then) G has an augmenting path with respect to M.*

Example 4.3. For the following graph drawn below, and the matching $M = \{\{1, C\}, \{3, F\}, \{5, A\}\}$, let's see which sequences of vertices are and are not alternating walks and augmenting paths.



|M| = 3

Sequence	Alternating Walk	Augmenting Path
$2, A, 5$		
$G, 4$		
$3, F, 4, C, 3$		
$1, C, 6$		
$6, F, 3, C, 1, B$		

5 Maximum Matching Algorithm

Like in a pure greedy strategy, we will try to grow our matching M one step at a time, building a sequence $M_0 = \emptyset, M_1, M_2, \ldots$, with $|M_k| = k$. However, to get M_k from M_{k-1} we will sometimes make use of an augmenting path (whose existence is guaranteed by Theorem 4.2) rather than just adding an edge. Specifically, we'll use the following three lemmas (two of which apply only to bipartite graphs, which is the case we will focus on):

Lemma 5.1 ("Certain shortest alternating walks are augmenting paths")**.** *Let G be bipartite, with bipartition (V_0, V_1),*[1] *and let M be a matching in G that is not of*

[1]Recall that a *bipartite* graph is a graph that is 2-colorable, and a *bipartition* of a bipartite graph is a partition of the vertex set $V = V_0 \cup V_1$ into the 2 color classes given by a 2-coloring. Thus all of the edges in the graph have one endpoint in V_0 and one endpoint in V_1.

maximum size. Let U be the vertices that are not matched by M, and $U_0 = V_0 \cap U$ and $U_1 = V_1 \cap U$. Then:

1.

2.

Lemma 5.2 ("Shortest alternating walks are easy to find")**.** *Finding shortest alternating walks in bipartite graphs reduces to*

Lemma 5.3 ("Augmenting paths can be used to efficiently grow matchings")**.** *Given a graph $G = (V, E)$, a matching M, and an augmenting path P with respect to M, we can*

Using these lemmas we will prove:

Theorem 5.4. Maximum Matching *can be solved in time $O(mn)$ on bipartite graphs with m edges and n vertices.*

We defer the proof of Lemmas 5.1 to the textbook.

Proof sketch of Lemma 5.2.

□

Proof of Lemma 5.3.

□

Example 5.5. We repeatedly apply Lemma 5.3 to our example matching from Example 4.3. We first consider the augmenting path $(G, 4)$ and grow our matching accordingly.

Next, we add the augmenting path $(6, F, 3, C, 1, B)$.

Note that $\{3, F\}$ and $\{1, C\}$ are not in the final matching, since they were in both the previous matching and the augmenting path (so they are "switched out" of the matching).

Putting it all together, we have the following algorithm:

```
MaxMatchingAugPaths(G):
Input  : A bipartite graph G = (V, E)
Output : A maximum-size matching M ⊆ E
Remove isolated vertices from G;
Let V_0, V_1 be the bipartition (i.e. 2-coloring) of V;
M = ∅;
repeat
    Let U be the vertices unmatched by M, U_0 = V_0 ∩ U, U_1 = V_1 ∩ U;
    Use BFS to find a shortest alternating walk P that starts in U_0 and
      ends in U_1;
    if P ≠ ⊥ then augment M using P via Lemma 5.3;
until P = ⊥;
return M
```

Algorithm 5.1: `MaxMatchingAugPaths()`